Numerical Performance Analysis of the Direct and Barnes-Hut Algorithm for Solving the N-Body Problem

J. Gabriel Balarezo[1][0009−0001−4386−2933] and Israel Pineda[2][1111−2222−3333−4444]

[1] School of Physical Sciences and Nanotechnology, Yachay Tech University, Urcuquí, Ecuador
juan.balarezo@yachaytech.edu.ec

[2] School of Mathematical and Computational Sciences, Yachay Tech University, Urcuquí, Ecuador
ipineda@yachaytech.edu.ec

Abstract. The N-body problem—which simulates the evolution of a system of particles interacting through a gravitational field—holds significant computational challenges due to its inherent complexity. In this work, we implemented and compared two approaches to N-body simulations—a direct method and the Barnes-Hut algorithm. The direct method computes pairwise gravitational forces between all particles (bodies), which results in an $\mathcal{O}(N^2)$ time complexity—ensuring accuracy but poorly scaling with large systems. Conversely, the Barnes-Hut algorithm leverages a hierarchical tree structure—quadtree for 2D and octree for 3D—to approximate distant forces, achieving a time complexity of $\mathcal{O}(N \log N)$ and improving performance for large-scale simulations. Both methods were implemented using `C++` programming language, combined with the `OpenMP` library for parallel processing and the `SDL2` graphics library for visualisation tools. Simulation results feature the trade-offs between computational cost and precision, with the direct method excelling in small systems and Barnes-Hut outperforming it as N increases. This work details the algorithms, their implementations, and performance comparisons, offering insights into their practical applications and limitations.

Keywords: N-Body simulation · Barnes-Hut algorithm · direct method · computational physics · parallel processing

1 Introduction

In physics, the N-body problem refers to the task of predicting the individual motions of a group of N interacting particles (bodies)—such as planets, stars, or subatomic particles—under the influence of mutual forces, typically gravity. Given a set of initial conditions (positions, velocities) and the laws governing the interactions (*e.g.*, Newton's law of gravitation), we are to determine the future state of the system. Applications of the N-body problem span various fields,

including astrophysics [5][1] (*e.g.*, simulating galaxy formation), molecular dynamics [4] (*e.g.*, simulating protein folding), and cosmology [3] (*e.g.*, simulating large-scale structure formation).

For the purposes of this report, we shall focus on the gravitational N-body problem. Let a number—N—of particles interact classically through Newton's law of gravitation. Then, the equations of motion for the i-th particle are given by [2]:

$$\frac{d^2\mathbf{r}_i}{dt^2} = -\sum_{j=1,j\neq i}^{N} \frac{Gm_j(\mathbf{r}_i - \mathbf{r}_j)}{|\mathbf{r}_i - \mathbf{r}_j|^3} \tag{1}$$

where $\mathbf{r}_i$ is the position vector of the i-th particle, G is the universal constant of gravitation, and m_i is the mass of the i-th particle. For $N = 1$ and $N = 2$ the equations can be solved analytically. On the other hand—for $N \geq 3$—there is no general analytical solution. Thereby, numerical methods are required to approximate a solution.

The main goal of this work is to carry out a numerical performance comparison between the direct method and the Barnes-Hut algorithm for solving the gravitational N-body problem. System sizes up to $N = 10^4$ were simulated, and the performance of both methods was evaluated in terms of the execution time of the force calculation routine. Since the time complexity of both methods is defined mainly by the number of operations required to compute the gravitational forces, the benchmark do not include the time required to integrate the equations of motion.

2 The Direct Method

Let's consider two interacting bodies with masses m_1 and m_2, and positions $\mathbf{r}_1$ and $\mathbf{r}_2$ respectively , see Figure 1. The gravitational force acting on m_1 due to m_2 is in the direction of $\boldsymbol{r}_2 - \boldsymbol{r}_1 = \boldsymbol{r}_{12}$, and is given by

$$\mathbf{F}_{12} = -\frac{Gm_1m_2}{|\mathbf{r}_{12}|^3}\mathbf{r}_{12} \tag{2}$$

Since the gravitational force is a symmetric interaction, the force acting on m_2 due to m_1 is given by

$$\mathbf{F}_{21} = -\mathbf{F}_{12} \tag{3}$$

Therefore, the total force $\mathbf{F}_i$ on body i, due to its interactions with the other $N - 1$ bodies is obtained by summing all the interactions

$$m_i\frac{d^2\mathbf{r}_i}{dt^2} = \sum_{j=1,j\neq i}^{N} \mathbf{F}_{ij} = -\sum_{j=1,j\neq i}^{N} \frac{Gm_im_j}{|\mathbf{r}_{ij}|^3}\mathbf{r}_{ij} \tag{4}$$

and dividing by m_i both sides we obtain Eq. 1, which describes the acceleration of the $i - th$ body due to the gravitational interactions with the other $N - 1$ bodies. For simulation purposes, the denominator $|\mathbf{r}_i - \mathbf{r}_j|$ is usually replaced

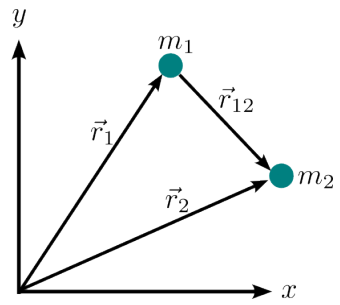

Fig. 1. A two-body system depicting the gravitational interaction between two particles with masses m_1 and m_2.

by $(|\mathbf{r}_i - \mathbf{r}_j|^2 + \epsilon^2)^{3/2}$—often referred to as the softening parameter ϵ—to avoid singularities when two particles are very close to each other.

3 The Barnes-Hut Algorithm

As previously mentioned, the direct method scales poorly with large systems—and hence—we need to find a more efficient way to compute the forces acting on each body. The Barnes-Hut algorithm is an approximation algorithm for performing an N-body simulation with a time complexity of $\mathcal{O}(N \log N)$. The main idea is to approximate long-range forces by replacing a group of distant bodies with their centre of mass. In exchange for a small loss of precision, this scheme significantly speeds up calculations. It utilises a hierarchical tree structure—a quadtree in 2D and an octree in 3D—to represent the spatial distribution of bodies and their interactions (see Figure 2).

The tree is built by recursively subdividing the simulation area into quadrants until each node contains either zero or one body—namely a leaf node. We then keep track of the size, total mass and centre of mass for each node, as these will be used to update the state of the system afterwards. These quantities are computed as follows

$$M = \sum_{i=1}^{n} m_i \tag{5}$$

$$\mathbf{r}_{cm} = \frac{1}{M} \sum_{i=1}^{n} m_i \mathbf{r}_i \tag{6}$$

where n is given by the number of bodies in the node.

Once the tree has been built and the centre of mass and total mass of each node have been updated, we proceed to compute the forces acting on each body. For this, we define a condition—the theta criterion—which determines whether a node is sufficiently far away from the body being considered, such that the approximation is valid. The theta criterion is defined in terms of the size of the node s and the distance d from the body to the centre of mass of the node,

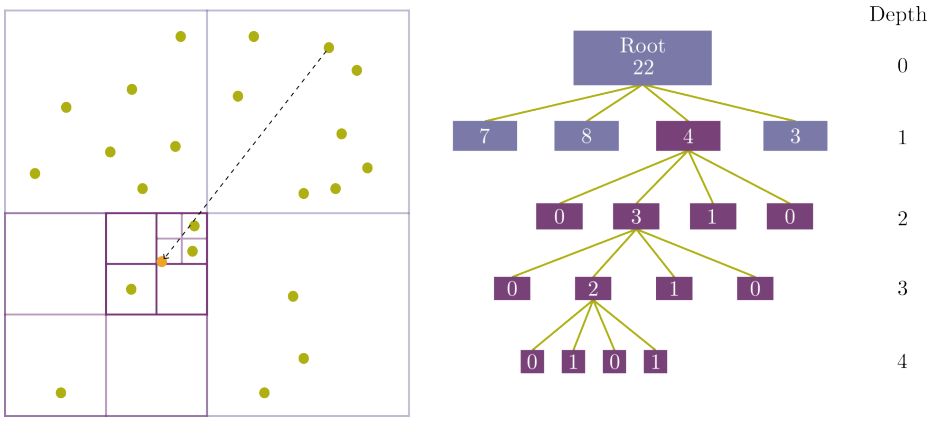

Fig. 2. A Barnes-Hut tree structure for a two-dimensional system of particles. The root node represents the entire simulation area, and each subsequent level divides the area into quadrants (four child nodes). If sufficiently far away, the node is treated as a single body with mass M and position $\boldsymbol{r}_{cm}$ (orange circle).

$s/d < \theta$. Here, theta is a parameter between 0 and 1 that determines the accuracy of the approximation, and in consequence, the speed of the algorithm. Should the condition be satisfied, the force acting on the body is computed as if the node were a single body with mass M and position $\mathbf{r}_{cm}$ (see Figure 2); otherwise, the algorithm recursively traverses the tree, visiting the child nodes until the condition is met or a leaf node is reached.

4 Simulation Setup

In this section we outline the main components of the implementation of the algorithms. A complete description of the implementation and the source code can be found in the GitHub repository https://github.com/quantum-gabo/N-Body-Simulation

4.1 Data Structures

The implementation of the direct method and the Barnes-Hut approximation requires the definition of some objects and data structures to represent the bodies, 2D vectors and the quadtree. Here below we list and describe the main data structures used in the implementation.

- `Body` class: represents a particle in the simulation. It stores the mass, radius, position, velocity, acceleration, previous position and a boolean flag to indicate if the body is a central body (*e.g.* a black hole).
- `Vector2D` class: represents a two-dimensional vector. It provides methods for vector addition, subtraction, multiplication by a scalar, and computing the

norm of the vector. This class is used to represent the position, velocity, and acceleration of bodies in the simulation.
- `Quadtree` class: integrates other two classes—`Node` and `Quad`—to represent the tree structure used in the Barnes-Hut algorithm. It provides methods for inserting bodies into the tree, updating the centre of mass and total mass of nodes, and traversing the tree to compute the forces. Once a parent node has been subdivided, its index is added to a vector `parents`, and the child nodes are stored in a vector `nodes`. Each parent node stores the index of its first child node, and each node stores the index of the following node used to guide the traversal of the tree.

4.2 Initial Conditions and Time Integration

Equation (4) is a second-order ordinary differential equation (ODE). Since we are working in 2D, the overall system consists of $2N$ second-order ODEs, and hence, require $4N$ initial conditions. These initial conditions include the two Cartesian components of position $\mathbf{r}_i$ and the two components of the velocity $\dot{\mathbf{r}}_i$ of each one of the N particles.

On the other hand, to simulate the system over time, we need to integrate the equations of motion so we can predict the state of the system at time $t + \Delta t$, given its state at time t. A variety of numerical integrators can be used, such as Euler, Verlet, or Runge-Kutta. Since we are simulating a physical system with many interacting particles, it is crucial to use a stable integrator. For this reason, we choose Verlet integration, as it is numerically stable, time-reversible, and conservative, making it well-suited for simulations of this nature [6]. The Verlet integration is defined as follows

$$\mathbf{r}_i(t + \Delta t) = 2\mathbf{r}_i(t) - \mathbf{r}_i(t - \Delta t) + \mathbf{a}_i(t)\Delta t^2 + \mathcal{O}(\Delta t^4) \tag{7}$$

where $\mathbf{a}_i(t)$ is the acceleration of the i-th body at time t and Δt is the time step. Moreover, since Verlet is not a self-starting method, the first step requires the use of a suitable approximation given by

$$\boldsymbol{r}_i(t_1) = \boldsymbol{r}_i(t_0) + \boldsymbol{v}_i(t_0)\Delta t + \frac{1}{2}\boldsymbol{a}_i(t_0)\Delta t^2 + \mathcal{O}(\Delta t^3) \tag{8}$$

this allows us to compute the positions of the bodies at the first time step where no previous position is available. Subsequent steps will use Eq. (8) to perform the integration.

5 Results and Discussion

In this section we present the results of the performance comparison between the direct method and the Barnes-Hut algorithm. Each system size was simulated for 100 time steps, and the execution time of each time step was recorded. Then, the average execution time was computed for each system size. The obtained

data is summarised in Table 1, which shows the average execution time for both methods, along with the speedup factor defined as the ratio between the execution time of the direct method and the Barnes-Hut algorithm. Figure 3 shows the performance comparison between the two methods, where the execution time is plotted in logarithmic scale against the number of particles.

Table 1. Execution time for varying system sizes using the direct method and Barnes-Hut approximation, along with the speedup factor.

Number of Particles	Direct Method (s)	Barnes-Hut (s)	Speedup
100	0.000048	0.000046	1.04
500	0.000877	0.000542	1.62
1000	0.003328	0.001538	2.16
2000	0.011730	0.001769	6.63
4000	0.054317	0.004049	13.41
8000	0.215674	0.009213	23.41
10000	0.320299	0.011732	27.29
15000	0.734168	0.018731	39.19
20000	1.310772	0.025587	51.23
30000	2.925272	0.049420	59.18
40000	5.189255	0.052630	98.61
50000	8.185188	0.069083	118.48
60000	11.801987	0.093106	126.75
80000	20.728767	0.113791	182.19
100000	32.540323	0.152524	213.36

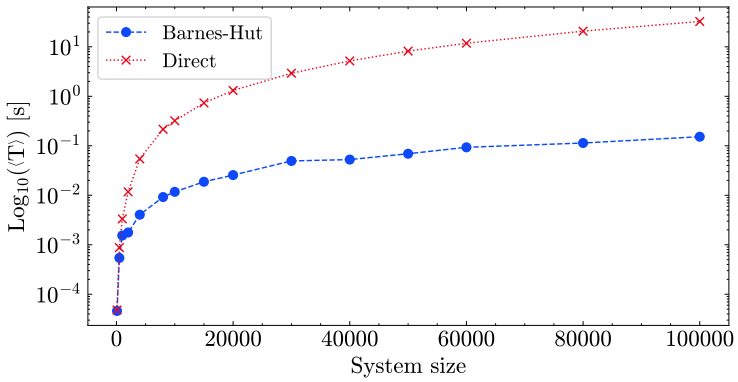

Fig. 3. Performance comparison between the direct method and the Barnes-Hut algorithm. The execution time is plotted in logarithmic scale against the number of particles. The direct method shows a quadratic scaling with the number of particles, while the Barnes-Hut algorithm exhibits a logarithmic scaling.

The obtained results show that the direct method and the Barnes-Hut algorithm have a similar performance for small systems—up to $N = 500$ particles—where the execution time is in the order of milliseconds. However, as the system size increases, the execution time of the direct method grows rapidly, reaching over 32 seconds for $N = 100K$ particles. On the other hand, the Barnes-Hut algorithm maintains a much lower execution time, reaching only 0.15 seconds for the same system size. The speedup factor increases significantly with the number of particles, reaching over 200 for $N = 100K$ particles. This demonstrates the effectiveness of the Barnes-Hut algorithm in reducing the computational cost of N-body simulations. Finally, parallel processing was used to speed up key operations, such as acceleration computation, integration, and vector resetting.

6 Conclusions

In this work, we have implemented and compared the performance of the direct method and the Barnes-Hut algorithm for solving the gravitational N-body problem. The former has a time complexity of $\mathcal{O}(N^2)$, while the latter achieves a time complexity of $\mathcal{O}(N \log N)$ by approximating distant forces using a hierarchical tree structure.

The results show that the direct method is suitable for small systems, whereas the Barnes-Hut algorithm outperforms it for larger systems, achieving a significant speedup factor over 200x for $N = 100K$ particles. This makes the Barnes-Hut algorithm a more efficient choice for N-body simulations, especially in astrophysical and cosmological applications where large systems are common. Finally, core components of the implementation were parallelised using the `OpenMP` library, which further improved the performance of both methods, without adding significant complexity to the code.

References

1. Chapter 31. Fast N-Body Simulation with CUDA (Apr 2025), https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-31-fast-n-body-simulation-cuda, [Online; accessed 6. Apr. 2025]
2. Heggie, D.: Gravitational n-body problem (classical). In: Françoise, J.P., Naber, G.L., Tsun, T.S. (eds.) Encyclopedia of Mathematical Physics, pp. 575–582. Academic Press, Oxford (2006). https://doi.org/https://doi.org/10.1016/B0-12-512666-2/00003-1, https://www.sciencedirect.com/science/article/pii/B0125126662000031
3. Klypin, A.: Methods for cosmological n-body simulations. URL http://www.skiesanduniverses. org/resources/KlypinNbody. pdf (2017)
4. Maksimenko, V., Lipnitskii, A., Kartamyshev, A., Poletaev, D., Kolobov, Y.R.: The n-body interatomic potential for molecular dynamics simulations of diffusion in tungsten. Computational Materials Science **202**, 110962 (2022). https://doi.org/https://doi.org/10.1016/j.commatsci.2021.110962, https://www.sciencedirect.com/science/article/pii/S0927025621006571

5. Wang, L., Iwasawa, M., Nitadori, K., Makino, J.: petar: a high-performance n-body code for modelling massive collisional stellar systems. Monthly Notices of the Royal Astronomical Society **497**(1), 536–555 (07 2020). https://doi.org/10.1093/mnras/staa1915, https://doi.org/10.1093/mnras/staa1915
6. William H. Press, Saul A. Teukolsky, W.T.V.B.P.F.: Numerical recipes: the art of scientific computing. Cambridge University Press, 3rd ed edn. (2007), http://gen.lib.rus.ec/book/index.php?md5=0edc79280d53ef968ffa05d63dfd5e55